

AgentCore

From a local SRE script to a production agent

Runtime → Memory → Code Interpreter → Observability → Identity/Gateway

Agenda

1. **The production wall** — why a working script isn't a product
2. **Runtime** — the serverless heart, one microVM per session
3. **Memory** — remember incidents across sessions
4. **Tools** — Code Interpreter & Gateway: more tools without writing them
5. **Observability** — see the agent think
6. **Identity** — auth at the edge, briefly
7. **Putting it together** + Q&A

0 · The production wall

A working script is not a product.

It works on your laptop. Now ship it.

You built the SRE agent in layer 04: it reasons about an incident, calls a tool, verifies the fix. On your machine, it works.

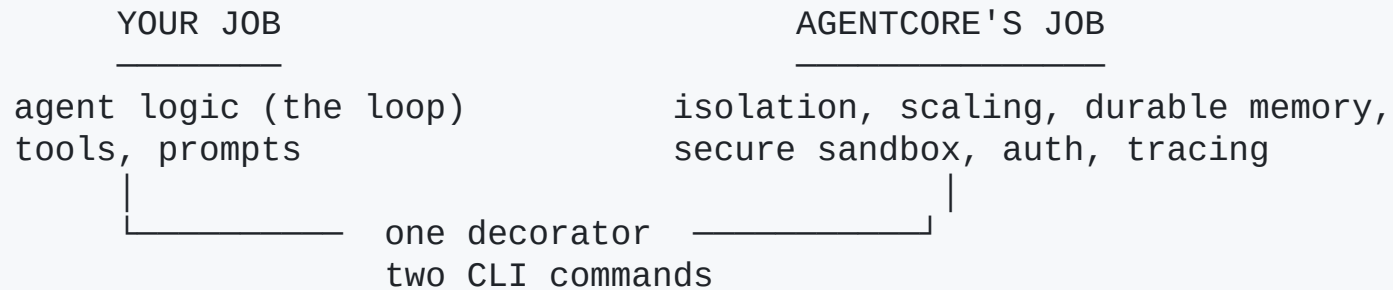
Then you try to ship it — and hit a **wall of infrastructure**:

- run for **many users at once** without leaking one incident into another
- **remember** past incidents across sessions
- run **untrusted code** safely
- **scale** on demand, be **observable** when it misbehaves at 3 a.m.

Building that from scratch is **months** of work. It's also not *agent* work — it's plumbing.

AgentCore = managed production capabilities

Amazon Bedrock AgentCore is a set of managed services that hand you those production capabilities, so you keep writing *agent logic*, not infrastructure.



It's **modular** — each service stands alone or composes. Switch on what you need; ignore the rest.

Where AgentCore sits in our stack

Each service maps almost one-to-one onto a concept we already built:

AgentCore service	What it does
Runtime	serverless host, one microVM per session
Memory	short- + long-term, cross-session
Code Interpreter	secure sandbox to run real code
Gateway	turn APIs/Lambda into MCP tools
Identity	inbound + outbound auth
Browser	cloud browser the agent drives
Observability	traces of reasoning, tools, memory

1 • Runtime

The serverless heart. Everything else is optional.

"Lambda, but for stateful agents"

Runtime is a serverless host **purpose-built for agents**:

	<u>AWS Lambda</u>	<u>AgentCore Runtime</u>
lifetime	one invocation, gone	a SESSION: many calls, up to 8h
state	stateless	keeps in-memory state between calls
isolation	per-invocation	per-session microVM (CPU/mem/fs)
max time	15 min	8 h
fit	a stateless function	a multi-turn, tool-using agent

A chat agent is **stateful and long-lived** — it holds a conversation. Lambda's stateless 15-min model doesn't fit. Runtime is the shape that does.

Runtime facts that change how you code

- **One microVM per `sessionId`**. Each session = a dedicated Firecracker microVM — isolated CPU/mem/fs. On termination it's destroyed, memory wiped → **no cross-session leakage**. (the enterprise-security headline)
- **Long-lived but ephemeral**. Up to **8 h**, killed after **15 min idle**.
State in the microVM is **not durable** → use **Memory** (part 3) for anything that must survive.
- **Framework- & model-agnostic**. LangChain, CrewAI, Strands; Claude, Nova, Gemini. AgentCore doesn't care.
- **Streaming, MCP, A2A** — powers the *interface* edge from layer 05.
- **Billed for active compute**, not wall-clock — an 8 h session that mostly waits on the model is not 8 h of billing.

The one line that makes a script deployable

The **only** difference between your local script and a cloud agent is a decorator:

```
from bedrock_agentcore.runtime import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

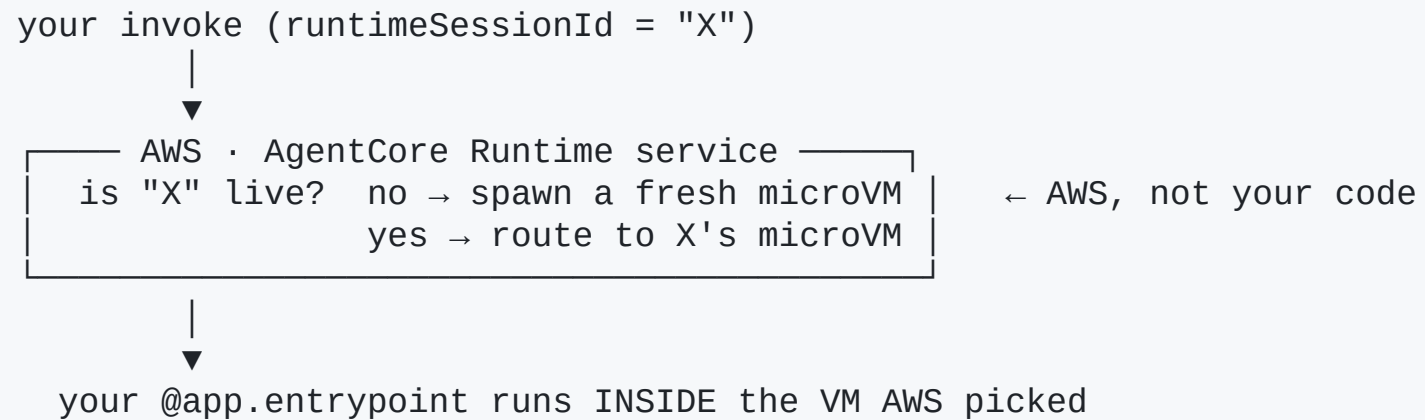
@app.entrypoint
def invoke(payload, context):    # payload = request JSON; context = session info
    ...
    return {"response": "..."}    # structured output back to the caller

if __name__ == "__main__":
    app.run()
```

`@app.entrypoint` tells Runtime how to call your agent. AgentCore handles the container, scaling, isolation, and HTTP plumbing. **That's the whole contract.**

Who spawns the microVM? (not you)

You never spawn a VM. **AWS does**, keyed on the `sessionId` you pass:



The `sessionId` is a **parameter of the API call**, not state in your code:

```
agentcore invoke '{"prompt":"..."}' --session-id incident-checkout-5xx-0000000000
```

Designing the sessionId — one session per incident

Rule: **one session = one conversation**. For an alert channel that means **one session per incident** — not per channel (alerts bleed together), not per message (no follow-up continuity).

```
def on_alert(alert):  
    # STABLE incident key from the alerting system's dedup/fingerprint  
    incident_id = f"incident-{{alert['service']}}-{{alert['fingerprint']}}".ljust(33, "0")  
    actor_id     = f"svc:{{alert['service']}}"                # durable identity; lessons accrue here  
    client.invoke_agent_runtime(  
        agentRuntimeArn=AGENT_ARN,  
        runtimeSessionId=incident_id,                        # new id → spawn; seen id → route  
        payload=json.dumps({"prompt": alert["text"]}).encode())
```

- **New incident** → new id → AWS spawns. **Update to a live one** → reuse id → routes to the warm VM, context intact.
- Derive the id from the **alert fingerprint** (≥33 chars, unguessable).

Incidents outlive VMs

An SRE may reply **30 min later** — after the 15-min idle timeout already killed the VM.

```
14:35  alert → spawn VM (session X)  —▶ agent investigates
14:52  (15 min idle) —————▶ VM TERMINATED, RAM wiped
15:20  SRE replies on the same incident —▶ reuse session X
      |
      |— Memory OFF → fresh empty VM, context lost
      |— Memory ON  → AgentCoreMemorySaver REPLAYS the thread → continuity
```

With Memory on, reusing the same `(session_id, actor_id)` replays the saved thread into a fresh VM. **Without it, a timed-out session loses everything.**

Runtime — remember

- Runtime = **serverless host for agents**: one **isolated microVM per session**, up to **8 h**, framework/model-agnostic, with streaming.
- **Stateful within a session** (unlike Lambda) but **ephemeral** — durable memory is a separate service.
- `@app.entrypoint` is the **only** code change from local → cloud.
- **AWS owns** the session → microVM lifecycle; the **caller picks the** `sessionId` (≥33 chars). One session per conversation/incident; the durable identity is `actor_id`.

2 · Memory

Remember incidents across sessions.

Within-session is not enough

The microVM holds state up to 8 h — but the moment the session ends, or a new incident opens next week, that context is **gone**.

```
Within-session state (microVM) — 04 "working memory" (RAM now), dies with VM
AgentCoreMemorySaver   (STM)   — exact conversation history, survives restarts
AgentCoreMemoryStore   (LTM)   — 04 "episodic": extracted facts, cross-session
```

AgentCore Memory is the durable layer — two LangGraph objects from

`langgraph-checkpoint-aws` :

- `AgentCoreMemorySaver` → **short-term**: persists this session's turns
- `AgentCoreMemoryStore` → **long-term**: extracts & searches cross-session facts

Four strategies — what gets remembered

LTM isn't one bucket. You turn on **strategies** that decide *what* to extract from the raw conversation — and they line up with the layer-04 memory types:

Strategy	Extracts	04 type	SRE example
Semantic	generalized facts	semantic	"checkout-api uses a DB connection pool"
Episodic	events (what/when/outcome)	episodic	"14:35 v2.3 shrank pool → 5xx → rollback fixed it"
User preference	how the actor likes things	profile	"the SRE team wants a short tabular report"
Summary	condensed session summaries	compression	"Incident #4821: 5xx from v2.3, rolled back"

Each runs **extract** → **consolidate** → **reflect**, driven by system prompts — **no code from you.**

Scoping memory — namespaces with placeholders

You mix strategies on one store; each writes to its own **namespace**.

`{actorId}` / `{sessionId}` are filled at runtime — that's how one store stays scoped per team and per incident:

```
memory = MemoryClient(region).create_memory_and_wait(  
    name="SREAgentMemory",  
    event_expiry_days=30,                                # STM retention knob  
    strategies=[  
        {"semanticMemoryStrategy": {"name": "Facts", "namespaces": ["/facts/{actorId}/"]}},  
        {"userPreferenceMemoryStrategy": {"name": "Prefs", "namespaces": ["/prefs/{actorId}/"]}},  
        {"summaryMemoryStrategy": {"name": "Summaries", "namespaces": ["/sum/{actorId}/{sessionId}/"]}},  
    ])
```

Shared namespace = shared across the team; `{actorId}`-keyed = **isolated** per actor. This is **type × scope** from layer 04, made concrete.

Wiring it in — identity keys at invoke time

```

checkpointer = AgentCoreMemorySaver(MEMORY_ID, region)    # short-term
store        = AgentCoreMemoryStore(MEMORY_ID, region)   # long-term
_agent = create_react_agent(model=llm, tools=[...], prompt=SYSTEM,
                             checkpointer=checkpointer,   # persists this session's turns
                             store=store)                # extracts/searches cross-session facts

@app.entrypoint
def invoke(payload, context):
    config = {"configurable": {
        "thread_id": context.session_id,    # → AgentCore session_id (this incident)
        "actor_id":  actor_id}              # → AgentCore actor_id   (the SRE team)
    }
    result = _agent.invoke({"messages": [("human", payload["prompt"])]}, config=config)
    return {"response": result["messages"][-1].content}

```

`thread_id` → `session_id`, `actor_id` → `actor_id`. The microVM is **pinned to one session** → build the agent **once** (lazy global) and reuse it all session.

Prove it — cross-session recall

```
# Session A: record the incident outcome
agentcore invoke '{"prompt":"Resolved: v2.3 shrank the DB pool, rolled back to v2.2."}' \
  --session-id incident-A-checkout-5xx-000000000000 --headers "Actor-Id:sre-team"
sleep 20                                     # LTM extraction is async

# Session B (NEW incident, same team): the lesson carries over
agentcore invoke '{"prompt":"checkout-api 5xx again – anything we learned before?"}' \
  --session-id incident-B-checkout-5xx-000000000000 --headers "Actor-Id:sre-team"
# → "Last time this was deploy v2.3 shrinking the DB pool; check the latest rollout first."
```

That's the **episodic memory** payoff from layer 04 — now a managed service, not hand-managed files or mem0.

Memory — remember

- Runtime gives memory **within** a session; **AgentCore Memory** is the **durable** layer (survives termination — this is what makes incidents outlive VMs).
- `AgentCoreMemorySaver` = STM (conversation replay) · `AgentCoreMemoryStore` = LTM (cross-session facts).
- **Four strategies** (semantic/episodic/preference/summary) = layer-04 memory types; extraction is **prompt-driven, managed**.
- **Namespaces** with `{actorId}` / `{sessionId}` = type × scope. Durable identity is the **actor**, not the session.

3 · Code Interpreter

Let the agent compute, not guess.

Don't ask a model to eyeball numbers

LLMs **pattern-match numbers** instead of calculating. "5xx spiked at 14:32" by eyeballing a timeseries is a **guess**.

Without Code Interpreter

model stares at numbers
→ "looks like ~14:32" (guess)
→ confident, possibly wrong

With Code Interpreter

model WRITES Python:
df.diff().idxmax() → exact changepoint
runs in sandbox → real result
→ "5xx step-changed at 14:32, +5.8pp"

Code Interpreter = a secure sandbox (its own Firecracker microVM, with `numpy` / `pandas` / `matplotlib`) where the agent **writes and runs real code** and reads the output back.

A generic tool — the agent writes the code

```
@tool
def execute_python(code: str, description: str = "") -> str:
    """Run Python in a secure sandbox and return stdout. Use for metric analysis,
    statistics, correlating the 5xx timeseries with deploy timestamps."""
    with code_session(REGION) as c:                                # spins up the sandbox microVM
        resp = c.invoke("executeCode",
                        {"code": code, "language": "python", "clearContext": False})
        for event in resp["stream"]:                                # state persists across calls
            return json.dumps(event["result"])
```

Add `execute_python` to `tools=[...]`. Now the agent pulls raw metrics with `query_metrics`, then **writes code** to find the exact changepoint and correlate it with rollout timestamps.

Generic (agent decides what code to write) vs **fixed-function**
(`rollback_deployment` = one hard-coded action).

4 · Observability

See the agent think.

The whole loop, as a trace

`agentcore launch` wires up Observability automatically — OpenTelemetry traces in CloudWatch. One incident trace shows the **entire layer-04 loop as spans**:

```
▼ Agent Invocation (1 incident)
  └─ Model call      reason: "5xx since 14:32, suspect a deploy"
  └─ Tool: query_metrics      (320 ms)
  └─ Tool: execute_python    changepoint = 14:32      (sandbox 1.1 s)
  └─ Tool: get_rollout_history v2.3 @ 14:30      (210 ms)
  └─ Memory: retrieve      "v2.3 caused this before"
  └─ Model call      decide: roll back to v2.2
  └─ Tool: rollback_deployment patched → v2.2      (180 ms)
  └─ Model call      verify: re-measure 5xx → 0.1% ✓
```

Reasoning, every tool call, memory ops, code runs, model latency — **all there**. It's the **review/ops** view of the loop, for free.

5 · Identity & Gateway

The software edge, as managed services.

Auth and tool catalogs

These are the layer-05 **software edge** as managed services:

- **Identity**  — **inbound auth** (who may invoke: IAM or OAuth via Cognito/Okta/Entra) + **outbound auth** (the agent reaching Slack/GitHub with managed OAuth/keys, so creds never sit in your code). The "auth at the edge" story from layer 05.
- **Gateway**  — turns your existing APIs/Lambda (or MCP servers) into **MCP tools** behind one endpoint. Instead of hand-writing the EKS tools, you'd front the cluster through a Gateway and let *any* MCP agent use them.

For the SRE agent: **Identity** safely exposes it to on-call (and lets it post to Slack); **Gateway** scales 3 hand-written tools → a governed catalog.

6 · Putting it together

The same incident, on production-grade infrastructure.

One incident, every component

```
14:35 PagerDuty → invoke_agent_runtime(session = this incident) [Runtime: own microVM]
|
|— retrieve "v2.3 caused this before" [Memory: episodic, x-session]
|— query_metrics + execute_python → 5xx step-changed at 14:32 [Code Interpreter: real compute]
|— get_rollout_history → v2.3 @ 14:30 is the suspect [tool on real EKS]
|— rollback_deployment → v2.2 [tool on real EKS]
|— re-measure 5xx → 0.1% , write the lesson to memory [review + Memory write]

every step traced in CloudWatch [Observability]
```

You wrote **agent logic**; isolation, memory, sandbox, scaling, auth, and tracing came as **managed services** — switched on one at a time.

Takeaways

1. A working script $\neq$ a product — the gap is **isolation, memory, sandbox, scaling, auth, tracing**. AgentCore hands you those as **managed services**.
2. **Runtime** is the core: serverless, **one isolated microVM per session**, 8 h; `@app.entrypoint` is the only code change. Durable identity is `actor_id`.
3. **Memory** = STM (saver) + cross-session LTM (store); four strategies = layer-04 memory types; `thread_id→session_id`, `actor_id→actor_id`.
4. **Code Interpreter** = secure sandbox so the agent **computes, not guesses** — a generic tool, distinct from fixed ones.
5. **Observability** = the whole loop as CloudWatch traces, automatically.
6. **Identity + Gateway** = the layer-05 software edge (auth + API $\rightarrow$ MCP).
7. The pattern: **build agent logic, switch on production capabilities** — keep what you need, ignore the rest.

Thank you

Questions?

Full notes, code & sources: this folder's `README.md`